

SQL Interview Recall Guide

Tricks, patterns and mnemonics for every question type in your assignment

Basics | Projection | Selection | Operators | MRF | Group By | Having | Sub Queries | Joins |
DDL/DML/TCL/DCL | SRF | Projects

The 10 Golden Rules (read these 5 minutes before the interview)

#	Rule	Memory hook
1	WAQTD = Write A Query To Display. First find: WHAT is asked (SELECT list) and WHAT is the condition (WHERE).	ASKED vs GIVEN
2	Clause order written: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY. Executed: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.	Written: S F W G H O. Run: F W G H S O
3	A single row cannot be KING and SCOTT at once. 'Name is KING and SCOTT' means OR / IN.	Same column + 'and' = OR
4	NULL is never = or <> anything. Use IS NULL / IS NOT NULL. Any math with NULL gives NULL, so use NVL.	NULL needs IS
5	WHERE filters ROWS before grouping. HAVING filters GROUPS after grouping. Aggregates are allowed only in HAVING / SELECT.	W = rows, H = groups
6	'in each', 'for each', 'wise' means GROUP BY. 'if / only if / at least' on a group means HAVING.	each = GROUP BY
7	Nth MAX: MAX with < and nest N-1 times. Nth MIN: MIN with > and nest N-1 times.	MAX goes with small sign
8	Sub query: ASKED goes outside, GIVEN goes inside. Link them with the common column.	Outside = asked
9	Join: n tables need n-1 join conditions. 'Even if' means outer join. Table to be shown fully decides LEFT or RIGHT.	Keep-all side = side name
10	Child table has the FK. Create/insert PARENT first. Delete/drop CHILD first.	Parent first in, last out

How to practise: for every question, say out loud: (1) which table(s)? (2) what is ASKED? (3) what is GIVEN / the condition? (4) is it row-level (WHERE), group-level (HAVING), or comparison with another row (sub query / self join)? That 4-step habit solves almost every question in your list.

PART A: Basic Theory (Q1-10) - one-liners to speak in the interview

Topic	Answer to remember
SEQUEL history	Developed at IBM in the early 1970s by Donald Chamberlin and Raymond Boyce, inspired by E.F. Codd's 1970 relational model paper. Original name SEQUEL (Structured English Query Language), shortened to SQL because of a trademark clash. Oracle released the first commercial SQL product (1979). ANSI standard in 1986, ISO in 1987.
Data	Raw facts and figures with no meaning on their own. Example: 7369, SMITH, 800. Meaningful processed data = information.
Database	Organised collection of related data stored so it can be easily accessed, managed and updated. Example: the company DB holding EMP, DEPT, SALGRADE tables.
DBMS	Software to create and manage a database. Data stored as files, no relationships between tables, no or weak constraints, single user, no Codd rules. Example: file system, XML.
RDBMS	DBMS where data is stored in tables (rows and columns) with relationships through keys. Supports constraints, multi-user, follows Codd's rules. Examples: Oracle, MySQL, SQL Server, PostgreSQL.
Table terms	Row = record = tuple. Column = field = attribute. Cardinality = number of rows. Degree = number of columns.

E.F. Codd's rules: 13 rules numbered 0 to 12

No.	Rule	No.	Rule
0	Foundation rule (must be relational)	7	High-level insert, update, delete
1	Information rule (all data in tables)	8	Physical data independence
2	Guaranteed access (table + PK + column)	9	Logical data independence
3	Systematic treatment of NULL	10	Integrity independence
4	Dynamic online catalog	11	Distribution independence
5	Comprehensive data sublanguage (SQL)	12	Non-subversion rule
6	View updating rule		

Mnemonic for Codd: 'Fine Information Gets Null Dynamic Comprehensive Views, High Physical Logical Integrity, Distribution Non-subversion' = Foundation, Information, Guaranteed, Null, Dynamic, Comprehensive, View, High-level, Physical, Logical, Integrity, Distribution, Non-subversion.

Datatypes (Oracle syntax)

Datatype	Syntax / example	Note
CHAR	gender CHAR(1)	Fixed length, pads with spaces, max 2000 bytes
VARCHAR2	ename VARCHAR2(20)	Variable length, uses only needed space, max 4000 bytes
NUMBER	sal NUMBER(8,2) id NUMBER(4)	(precision, scale). 8,2 = 6 digits + 2 decimals
DATE	hiredate DATE	Stores date + time. Default format DD-MON-YY
TIMESTAMP	created TIMESTAMP	Date with fractional seconds
CLOB / BLOB	notes CLOB photo BLOB	Large text / large binary (images, files)
Others	INT, FLOAT, LONG, RAW	INT maps to NUMBER(38)

CHAR	VARCHAR / VARCHAR2
Fixed length. 'AB' in CHAR(5) uses 5 bytes (padded).	Variable length. 'AB' in VARCHAR2(5) uses 2 bytes.
Faster for fixed-size data (gender, pincode).	Saves memory for names, addresses, emails.
Max 2000 bytes.	Max 4000 bytes.

Constraints with syntax

```
CREATE TABLE emp2 (
  empno  NUMBER(4)    PRIMARY KEY,           -- unique + not null, only ONE per table
  ename   VARCHAR2(20) NOT NULL,              -- value required
  email   VARCHAR2(40) UNIQUE,                -- no duplicates, NULL allowed
  sal     NUMBER(8,2) CHECK (sal > 0),         -- condition must be true
  city    VARCHAR2(20) DEFAULT 'PUNE',        -- value used when none given
  deptno  NUMBER(2)    REFERENCES dept(deptno) -- FOREIGN KEY, must exist in parent PK
);
-- named table-level style
```

CONSTRAINT pk_emp PRIMARY KEY (empno), CONSTRAINT fk_d FOREIGN KEY (deptno) REFERENCES dept(deptno)

PRIMARY KEY	FOREIGN KEY
Uniquely identifies each row. No duplicate, no NULL.	Links child table to parent table's PK. Duplicates and NULL allowed.
Only one per table.	Many per table.
Defined on the parent table.	Defined on the child table.

UNIQUE (constraint)	DISTINCT (keyword)
Applied while creating or altering table (DDL).	Used in SELECT to remove duplicate rows from the OUTPUT.
Prevents duplicates from being stored. Allows NULLs.	Does not change stored data, only the display.

PART B: Question Decoder - words in the question tell you the code

If the question says...	Write this	Example
all the details / complete info	SELECT *	SELECT * FROM emp
name / designation / joining date / reporting manager number	ename / job / hiredate / mgr	'job as designation' = job AS designation
no repeated records / different / unique values	DISTINCT (applies to the whole column combination)	SELECT DISTINCT job, deptno FROM emp
annual salary / half term / quarter term	sal*12 / sal*6 / sal*3	sal*12 AS annual
10% hike / 20% deduction	sal + sal*0.10 or sal*1.10 / sal - sal*0.20 or sal*0.80	sal*1.12 AS hike12
Rs 100 penalty / Rs 50 bonus	sal - 100 / sal + 50	sal + 50 AS bonus
total salary to pay (with commission)	sal + NVL(comm,0)	SUM(sal + NVL(comm,0))
alias / 'as'	AS name (use "double quotes" for spaces)	sal*12 AS "Annual Sal"
who / whose / for / if / working as / earning	WHERE condition	WHERE job='MANAGER'
not / except / other than	<> or NOT	WHERE deptno <> 20
in dept 10 and 20 / names are A and B (one column)	IN (...) or OR. Never AND on the same column.	deptno IN (10,20)
working as manager in dept 20 (different columns)	AND	job='MANAGER' AND deptno=20
between X to Y (inclusive)	BETWEEN low AND high (low first)	sal BETWEEN 1000 AND 5000
more than X but less than Y	> X AND < Y (exclusive) unless 'between'	sal > 1000 AND sal < 5000
earning commission / not earning / no manager	IS NOT NULL / IS NULL	comm IS NULL, mgr IS NULL
name starts / ends / has / 2nd letter	LIKE with % and _	see Part C
number of / how many / total / average / max / min	COUNT / SUM / AVG / MAX / MIN (MRF)	COUNT(*)
in each / for each / wise	GROUP BY that column	GROUP BY deptno
if / only if / at least / exceeds (on groups)	HAVING aggregate condition	HAVING COUNT(*) >= 2
more than ADAMS / same dept as JONES (value of a person)	Sub query Case 1	sal > (SELECT sal ...)
dname / loc of employee... or employees of dept ACCOUNTING	Sub query Case 2 or Join	IN (SELECT deptno ...)
more than all / than any / at least one	> ALL / > ANY	see Part F
2nd / 3rd / Nth max or min	Nested sub query or DENSE_RANK	see Part F
manager of / reporting to / manager's manager	mgr and empno link (self relation)	see Part F and G
even if / not assigned / no employees	Outer join (LEFT / RIGHT / FULL)	see Part G
ename and dname together (two tables in output)	Inner join (needs both tables in SELECT)	emp e JOIN dept d ON ...

PART C: Operators, LIKE patterns, Dates

Group	Operators	Memory
Arithmetic	+ - * /	Used in projection (annual sal, hike)
Comparison	= > < >= <= <> != ^=	'Not equal' has 3 forms
Logical	AND OR NOT	Priority: NOT, then AND, then OR. Use brackets.
Special	IN, NOT IN, BETWEEN, LIKE, IS NULL, IS NOT NULL	I B L N = In, Between, Like, Null
Sub query	ALL, ANY / SOME, EXISTS	Used with multi-row sub queries
Concatenation		ename ' - ' job

Trap list: (1) BETWEEN is inclusive and the smaller value comes first. (2) 'Hired after 1981' means hiredate > DATE '1981-12-31'. 'Hired before 1987' means hiredate < DATE '1987-01-01'. 'Hired in 1981' means BETWEEN DATE '1981-01-01' AND DATE '1981-12-31' (or TO_CHAR(hiredate,'YYYY')='1981'). (3) 'Salary is 3000 and 5000' means IN (3000,5000). (4) 'Not working in dept 10' keeps rows where deptno is NULL out too, because NULL <> 10 is not true.

LIKE patterns: % = any number of characters (even 0), _ = exactly one character

Question	Pattern
Name starts with S / ends with R / contains A	'S%' '%R' '%A%'
Exactly 2 A's in name	LENGTH(ename) - LENGTH(REPLACE(ename,'A')) = 2
Consecutive A (AA) / two times A anywhere	'%AA%' / '%A%A%'
2nd character M / last 2nd character E	'_M%' / '%E_'
Starts with M, 4th char L, ends with R	'M__L%R' (two underscores = chars 2 and 3)
Exactly 4 characters / 5 characters	'____' (or LENGTH(ename)=4)
Starts with vowel	REGEXP_LIKE(ename,'^[AEIOU]') or ename LIKE 'A%' OR ename LIKE 'E%' ...
Starts with consonant / ends with vowel	NOT REGEXP_LIKE(ename,'^[AEIOU]') / REGEXP_LIKE(ename,'[AEIOU]\$')
Name starts with the real symbol % (and ends with _)	LIKE '%%%_' ESCAPE '\' (escape char makes % or _ literal)
Name has the symbol % / starts with % having 4th char _ and ends with %	LIKE '%\%%%' ESCAPE '\' / LIKE '%___%\%' ESCAPE '\' (check your exact pattern)
Salary starts with 1 / 4 digit salary	TO_CHAR(sal) LIKE '1%' / sal BETWEEN 1000 AND 9999 or LENGTH(sal)=4
Salary ends with 50	TO_CHAR(sal) LIKE '%50' (or MOD(sal,100)=50)

Date tricks (use TO_CHAR for month, year, day)

```

WHERE TO_CHAR(hiredate,'MON') = 'DEC'                -- hired in December
WHERE TO_CHAR(hiredate,'YYYY') IN ('1981','1982','1987') -- hired in these years
WHERE TO_CHAR(hiredate,'DAY') LIKE 'WEDNESDAY%'      -- DAY pads with spaces, use TRIM or LIKE
WHERE TO_CHAR(hiredate,'MON') IN ('DEC','JAN','FEB')
WHERE hiredate = DATE '1981-12-03'                  -- date literal, safest form

```

PART D: Projection, Selection and Logical/Special operator samples (Q11-130)

Concept	Meaning	Syntax
Projection	Choosing COLUMNS (vertical slice). Uses SELECT list.	SELECT ename, sal FROM emp;
Selection	Choosing ROWS (horizontal slice). Uses WHERE.	SELECT * FROM emp WHERE deptno=10;
Both together	Columns from filtered rows	SELECT ename, sal FROM emp WHERE sal > 1000;

```
-- Q26-33 expressions
SELECT sal*12 AS annual, sal*6 AS half_term, sal*3 AS quarter_term FROM emp;
SELECT deptno, job, sal*1.10 AS hike10 FROM emp;
SELECT emp.*, sal*12 AS annual FROM emp; -- all details + one extra column
SELECT job AS designation, hiredate AS joining_date, mgr AS reporting_mgr FROM emp;
```

```
-- Q41-60 selection
SELECT * FROM emp WHERE sal > 1000;
SELECT ename, hiredate FROM emp WHERE hiredate = DATE '1981-01-01';
SELECT emp.*, sal*1.2 AS hike FROM emp WHERE hiredate > DATE '1985-12-31';
SELECT emp.*, sal*6 AS half_term FROM emp WHERE sal*12 > 14000;
```

```
-- Q61-80 logical
SELECT * FROM emp WHERE deptno IN (10,20); -- NOT deptno=10 AND deptno=20
SELECT * FROM emp WHERE job IN ('MANAGER','SALESMAN');
SELECT * FROM emp WHERE job='MANAGER' AND deptno IN (10,20) AND comm IN (400,1600);
SELECT ename AS a, deptno AS b, job AS c FROM emp WHERE comm > 100 AND deptno IN (10,20,30);
SELECT ename, sal*12 AS annual FROM emp
WHERE sal*12 > 800 AND deptno=30 AND job='SALESMAN' AND hiredate < DATE '1988-01-01';
```

```
-- Q104-110 NULL questions
SELECT emp.*, sal*3 AS three_months FROM emp WHERE sal IS NOT NULL AND mgr IS NULL;
SELECT sal*12 AS annual FROM emp WHERE comm IS NULL;
SELECT emp.*, comm*12 AS annual_comm FROM emp WHERE deptno=10 AND mgr IS NULL;
SELECT ename, sal, comm FROM emp WHERE comm IS NOT NULL AND deptno=30 AND job='SALESMAN';
SELECT ename, sal, job FROM emp
WHERE job='SALESMAN' AND deptno=30 AND sal IS NOT NULL AND comm IS NULL
AND hiredate BETWEEN DATE '1982-01-01' AND DATE '1988-12-31';
```

Special operator version (Q81-103): same questions as logical operators, but rewrite with IN instead of OR, and BETWEEN instead of >= AND <=. Example: *sal BETWEEN 1000 AND 5000 AND deptno IN (10,20)*.

PART E: MRF, GROUP BY, HAVING (Q131-165)

	SRF (Single Row Function)	MRF (Multi Row / Group / Aggregate)
Output	One output per input row	ONE output for MANY rows
Examples	UPPER, LENGTH, SUBSTR, NVL, ROUND	MAX, MIN, SUM, AVG, COUNT
NULL	Handles NULL (NVL)	Ignores NULL (except COUNT(*))
WHERE	Allowed	NOT allowed in WHERE (use HAVING)
Mix with columns	Allowed	Not with normal columns unless GROUP BY

MRF mnemonic: 'MAMCS' = MAX, AVG, MIN, COUNT, SUM. **COUNT(*)** counts all rows. **COUNT(comm)** counts non-NULL comm only. **Golden GROUP BY rule:** every non-aggregate column in SELECT must be in GROUP BY.

```
SELECT MAX(sal) FROM emp; -- Q131
SELECT MAX(comm), MIN(comm) FROM emp;
SELECT SUM(sal) FROM emp WHERE job='SALESMAN'; -- total to pay salesmen
SELECT COUNT(*) FROM emp WHERE job='MANAGER' AND deptno=20;
SELECT SUM(sal), AVG(sal) FROM emp WHERE hiredate BETWEEN DATE '1981-01-01' AND DATE '1987-12-31';
SELECT SUM(sal) FROM emp WHERE mgr = 7839;
SELECT COUNT(*) FROM emp WHERE job='PRESIDENT' AND deptno=10 AND sal > 4000;

-- GROUP BY (Q146-155): 'in each' means GROUP BY
SELECT deptno, COUNT(*) FROM emp WHERE job <> 'PRESIDENT' GROUP BY deptno;
SELECT job, SUM(sal) FROM emp GROUP BY job;
SELECT deptno, AVG(sal) FROM emp WHERE deptno <> 20 GROUP BY deptno;
SELECT job, COUNT(*) FROM emp WHERE ename LIKE '%A%' GROUP BY job;
SELECT deptno, COUNT(*), AVG(sal) FROM emp WHERE sal > 2000 GROUP BY deptno;
SELECT deptno, SUM(sal), COUNT(*) FROM emp WHERE job='SALESMAN' GROUP BY deptno;
SELECT sal, COUNT(*) FROM emp GROUP BY sal; -- how many times each salary appears

-- HAVING (Q156-165)
SELECT deptno, COUNT(*) FROM emp GROUP BY deptno
HAVING SUM(CASE WHEN job='CLERK' THEN 1 ELSE 0 END) >= 2; -- count all, but need 2+ clerks
```

```

SELECT deptno, SUM(sal) FROM emp GROUP BY deptno HAVING COUNT(*) >= 4;
SELECT job, COUNT(*), SUM(sal) FROM emp WHERE sal > 1200 GROUP BY job HAVING SUM(sal) > 3800;
SELECT deptno, COUNT(*) FROM emp WHERE job='MANAGER' GROUP BY deptno HAVING COUNT(*) = 2;
SELECT job, MAX(sal) FROM emp GROUP BY job HAVING MAX(sal) > 2600;
SELECT sal FROM emp GROUP BY sal HAVING COUNT(*) > 1; -- repeated salaries
SELECT hiredate FROM emp GROUP BY hiredate HAVING COUNT(*) > 1; -- duplicated hiredate
SELECT deptno FROM emp GROUP BY deptno HAVING AVG(sal) < 3000;
SELECT deptno FROM emp WHERE ename LIKE '%A%' OR ename LIKE '%S%' GROUP BY deptno HAVING COUNT(*) >= 3;
SELECT job, MIN(sal), MAX(sal) FROM emp GROUP BY job HAVING MIN(sal) > 1000 AND MAX(sal) < 5000;

```

Advanced HAVING: the word ALL becomes MIN / MAX / COUNT(DISTINCT)

Question pattern	Trick	Code in HAVING
All employees earn MORE than 3000	Smallest must beat it	MIN(sal) > 3000
All employees earn LESS than 2000	Largest must be below it	MAX(sal) < 2000
All employees are MANAGER (or ANALYST)	MIN = MAX = value	MIN(job)='MANAGER' AND MAX(job)='MANAGER'
All have the same hiredate / same dept	Only one distinct value	COUNT(DISTINCT hiredate) = 1
Job done only in one department	GROUP BY job	COUNT(DISTINCT deptno) = 1
All joined in 1981	Earliest and latest in range	MIN(hiredate) >= DATE '1981-01-01' AND MAX(hiredate) <= DATE '1981-12-31'
All have NULL comm	COUNT(col) ignores NULL	COUNT(comm) = 0
Dept count equals total clerks in company	Compare with a sub query	COUNT(*) = (SELECT COUNT(*) FROM emp WHERE job='CLERK')
Generic 'all satisfy a condition'	Count of matches = count of all	COUNT(*) = COUNT(CASE WHEN cond THEN 1 END)

PART F: Sub Queries (Q166-214 and Nested/Manager assignments)

Type	Meaning	Operators
SQSR (Single Query Single Row)	Inner query returns ONE value	= > < >= <= <>
MRSQ (Multi Row Sub Query)	Inner query returns MANY values	IN, NOT IN, ANY, ALL
Nested sub query	Sub query inside sub query	N-1 levels for Nth value
Co-related	Inner query uses a column of the outer query	Runs once per outer row

Case 1 (same table, compare with a person/value): inner and outer both on EMP. 'More than ADAMS' means `sal > (SELECT sal FROM emp WHERE ename='ADAMS')`.

Case 2 (two tables): ASKED goes outside, GIVEN goes inside. 'DNAME of SMITH': `asked = dname (DEPT, outer), given = SMITH (EMP, inner), link = deptno`.

Rule of thumb: inner column must have the same meaning and datatype as the outer column being compared. If the inner can return many rows, use IN instead of =.

```
-- Case 1
SELECT ename FROM emp WHERE sal > (SELECT sal FROM emp WHERE ename='ADAMS');
SELECT ename, deptno FROM emp WHERE deptno = (SELECT deptno FROM emp WHERE ename='JONES');
SELECT empno, ename, sal*12 FROM emp WHERE sal*12 > (SELECT sal*12 FROM emp WHERE ename='WARD');
SELECT ename, hiredate FROM emp
  WHERE hiredate < (SELECT hiredate FROM emp WHERE ename='SCOTT')
    AND hiredate > (SELECT hiredate FROM emp WHERE job='PRESIDENT');
SELECT ename, sal FROM emp WHERE sal > (SELECT sal FROM emp WHERE ename='MILLER')
    AND sal < (SELECT sal FROM emp WHERE ename='ALLEN');
SELECT ename, comm FROM emp WHERE comm IS NOT NULL AND job = (SELECT job FROM emp WHERE ename='SMITH');

-- Case 2 (asked outside, given inside)
SELECT dname FROM dept WHERE deptno = (SELECT deptno FROM emp WHERE ename='SMITH');      -- Q186
SELECT dname, loc FROM dept WHERE deptno = (SELECT deptno FROM emp WHERE ename='KING');
SELECT ename FROM emp WHERE deptno = (SELECT deptno FROM dept WHERE dname='ACCOUNTING'); -- Q191
SELECT ename, sal FROM emp WHERE deptno = (SELECT deptno FROM dept WHERE loc='CHICAGO');
SELECT * FROM emp WHERE job='SALESMAN' AND deptno = (SELECT deptno FROM dept WHERE dname='SALES');

-- Case 1 and 2 together (Q195-204)
SELECT ename FROM emp WHERE sal > (SELECT sal FROM emp WHERE ename='SCOTT')
    AND deptno = (SELECT deptno FROM dept WHERE dname='ACCOUNTING');
SELECT * FROM emp WHERE deptno IN (SELECT deptno FROM dept WHERE dname LIKE '%S');      -- Q200
SELECT dname, loc FROM dept WHERE deptno IN (SELECT deptno FROM emp WHERE sal = 800);    -- Q202
SELECT dname FROM dept WHERE deptno IN (SELECT deptno FROM emp WHERE comm IS NOT NULL);  -- Q203
SELECT dname FROM dept WHERE deptno IN (SELECT deptno FROM emp WHERE ename LIKE '%A%');  -- Q201
```

ALL / ANY: one table to remember forever

Written as	Same as	Easy words
> ALL (list)	> MAX(list)	Greater than every one = beat the biggest
> ANY (list)	> MIN(list)	Greater than at least one = beat the smallest
< ALL (list)	< MIN(list)	Less than every one = below the smallest
< ANY (list)	< MAX(list)	Less than at least one = below the biggest
= ANY (list)	IN (list)	Equal to one of them
<> ALL (list)	NOT IN (list)	Different from every one

Hook: 'ALL is strict and picks the extreme in that direction. ANY is lenient and picks the opposite extreme.' Also: **hired AFTER** all means `hiredate > ALL`. **hired BEFORE at least one** means `hiredate < ANY`. Dates behave like numbers: later date = bigger.

```
SELECT ename FROM emp WHERE sal > ANY (SELECT sal FROM emp WHERE job='SALESMAN');      -- Q205 (more than the salesman)
SELECT * FROM emp WHERE hiredate > ALL (SELECT hiredate FROM emp WHERE job='CLERK');    -- Q206
SELECT ename, hiredate FROM emp WHERE hiredate < ALL (SELECT hiredate FROM emp WHERE job='MANAGER'); -- Q208
SELECT ename FROM emp WHERE hiredate > ALL (SELECT hiredate FROM emp WHERE job='MANAGER')
    AND sal > ALL (SELECT sal FROM emp WHERE job='CLERK');      -- Q209
SELECT * FROM emp WHERE job='CLERK' AND hiredate < ANY (SELECT hiredate FROM emp WHERE job='SALESMAN'); -- Q210
SELECT * FROM emp WHERE deptno IN (SELECT deptno FROM dept WHERE dname IN ('ACCOUNTING','SALES')); -- Q211
SELECT dname FROM dept WHERE deptno IN (SELECT deptno FROM emp WHERE ename IN ('SMITH','KING','MILLER')); -- Q212
SELECT * FROM emp WHERE deptno IN (SELECT deptno FROM dept WHERE loc IN ('NEW YORK','CHICAGO')); -- Q213
SELECT ename FROM emp WHERE hiredate > ALL (SELECT hiredate FROM emp WHERE deptno=10);  -- Q214
```

Nth MAX / MIN: the ladder trick

Imagine salaries as a ladder. **MAX** stands at the top and looks DOWN, so it uses the < (small) sign. **MIN** stands at the bottom and looks UP, so it uses the > (big) sign.

Nth value = N-1 nested sub queries. Count the SELECT MAX words: that number is N. 2nd = 2 SELECTs, 5th = 5 SELECTs.

```

-- 2nd MAX                                -- 2nd MIN
SELECT MAX(sal) FROM emp                  SELECT MIN(sal) FROM emp
WHERE sal < (SELECT MAX(sal) FROM emp);    WHERE sal > (SELECT MIN(sal) FROM emp);

-- Need the employee? wrap it: WHERE sal = ( ...Nth value query... )
SELECT ename FROM emp WHERE sal = (SELECT MAX(sal) FROM emp
                                   WHERE sal < (SELECT MAX(sal) FROM emp
                                                WHERE sal < (SELECT MAX(sal) FROM emp))); -- 3rd max

-- Need DNAME? one more wrapper: SELECT dname FROM dept WHERE deptno IN (SELECT deptno FROM emp WHERE sal = (...));

-- Dates work the same: 2nd hired = MIN(hiredate) with >, hired before last = MAX(hiredate) with <

-- Alternative for ANY N (one line change, best for interviews)
SELECT * FROM (SELECT e.*, DENSE_RANK() OVER (ORDER BY sal DESC) rnk FROM emp e) WHERE rnk = 3; -- use ASC for Nth min

-- Co-related version (Nth max = N-1 salaries are bigger)
SELECT * FROM emp a WHERE 2 = (SELECT COUNT(DISTINCT sal) FROM emp b WHERE b.sal > a.sal); -- 3rd max

```

Emp-Manager relation: UP or DOWN

Direction	Idea	Code
UP: find the manager OF X	X's MGR column holds the boss's EMPNO	WHERE empno = (SELECT mgr FROM emp WHERE ename='X')
DOWN: employees reporting TO X	Employees whose MGR equals X's EMPNO	WHERE mgr = (SELECT empno FROM emp WHERE ename='X')
Manager's manager	Go UP twice	empno = (SELECT mgr FROM emp WHERE empno = (SELECT mgr FROM emp WHERE ename='X'))
Same boss (siblings)	Same MGR value	WHERE mgr = (SELECT mgr FROM emp WHERE ename='BLAKE')

Hook: 'UP = SELECT MGR, DOWN = SELECT EMPNO'. Whatever you SELECT inside is matched against the opposite column outside: inside MGR matches outside EMPNO; inside EMPNO matches outside MGR. For 'dname / loc of the manager', add a final DEPT layer: *SELECT dname FROM dept WHERE deptno = (SELECT deptno FROM emp WHERE empno = (SELECT mgr FROM emp WHERE ename='JONES'))*.

PART G: Joins (Theory 121-130, Inner 131-150, Outer/Self 151-170, Scenarios)

Join	Returns	Syntax (ANSI)
CROSS / Cartesian	Every row with every row. Rows = m x n. No condition.	FROM emp CROSS JOIN dept
INNER (equi)	Only matching rows from both tables	FROM emp e JOIN dept d ON e.deptno = d.deptno
NATURAL	Inner join on the same-named column automatically	FROM emp NATURAL JOIN dept (do not alias the common column)
LEFT OUTER	ALL rows of left table + matching right (else NULL)	FROM emp e LEFT JOIN dept d ON ...
RIGHT OUTER	ALL rows of right table + matching left (else NULL)	FROM emp e RIGHT JOIN dept d ON ...
FULL OUTER	ALL rows of both tables (LOJ UNION ROJ)	FROM emp e FULL JOIN dept d ON ...
SELF	Table joined with itself using two aliases	FROM emp e JOIN emp m ON e.mgr = m.empno

Joins vs sub queries: use a sub query when the output columns come from ONE table only (the other table just gives the condition). Use a join when output needs columns from TWO tables (ename AND dname). **Self join advantages:** shows employee and manager data side by side, and one join can replace several nested sub queries.

Inner join advantages: fast, returns only related data. **Disadvantages:** unmatched rows are lost, and wrong or missing ON condition gives a Cartesian product.

Inner join pattern (Q131-150)

```

SELECT e.ename, d.dname FROM emp e JOIN dept d ON e.deptno = d.deptno; -- Q131
SELECT d.dname, e.sal FROM emp e JOIN dept d ON e.deptno = d.deptno WHERE e.job='MANAGER';
SELECT e.*, d.loc FROM emp e JOIN dept d ON e.deptno=d.deptno
WHERE e.sal > 2000 AND d.dname='ACCOUNTING';
SELECT e.ename, d.deptno, d.loc FROM emp e JOIN dept d ON e.deptno=d.deptno
WHERE REGEXP_LIKE(e.ename, '[AEIOU]$'); -- name ends with vowel
SELECT e.ename, d.dname FROM emp e JOIN dept d ON e.deptno=d.deptno
WHERE TO_CHAR(e.hiredate, 'MON')='FEB' AND d.dname='ACCOUNTING';
SELECT e.ename, d.dname FROM emp e JOIN dept d ON e.deptno=d.deptno
WHERE e.sal > (SELECT sal FROM emp WHERE ename='SMITH')
AND e.sal < (SELECT sal FROM emp WHERE ename='KING')
AND e.deptno=30 AND d.dname='SALES'; -- joins + sub query mix

```

Outer joins: the 'even if' decoder

Words in question	Meaning	Join
'all employees even if not in any dept'	Employee table must be shown fully	emp LEFT JOIN dept
'all departments even if no employees'	Dept table must be shown fully	emp RIGHT JOIN dept (or dept LEFT JOIN emp)
'both: employees without dept AND depts without employees'	Both sides fully	emp FULL JOIN dept
'employees NOT working in any dept' (only those)	Outer join then keep unmatched	LEFT JOIN ... WHERE d.deptno IS NULL
'departments in which NO employees work' (only those)	Outer join then keep unmatched	RIGHT JOIN ... WHERE e.empno IS NULL

Hook: 'Even if' = keep everything (plain outer join). 'Not working / no employees / only those' = outer join + IS NULL filter on the other table's key (anti-join). **LEFT keeps the table written on the left of JOIN; RIGHT keeps the one on the right.** A WHERE filter on the NON-kept table turns an outer join into an inner join; put such a filter in ON instead.

```

SELECT e.ename, d.dname FROM emp e LEFT JOIN dept d ON e.deptno=d.deptno WHERE d.deptno IS NULL; -- Q151/154
SELECT e.*, d.* FROM emp e RIGHT JOIN dept d ON e.deptno=d.deptno WHERE e.empno IS NULL; -- Q152/155
SELECT e.*, d.* FROM emp e FULL JOIN dept d ON e.deptno=d.deptno; -- Q153/156
-- Added question: salesmen earning > 1000 even if no dept
SELECT e.ename, d.loc, e.sal FROM emp e LEFT JOIN dept d ON e.deptno=d.deptno
WHERE e.job='SALESMAN' AND e.sal > 1000; -- filters are on the kept table (safe)
-- All depts in NEW YORK even with no employees
SELECT e.ename, d.loc, d.dname, e.sal FROM emp e RIGHT JOIN dept d ON e.deptno=d.deptno WHERE d.loc='NEW YORK';

```

Self join: every 's' adds another alias

Phrase	Alias to use	Join condition
employee's ...	e	(base table)
manager's ... / reporting to	m	e.mgr = m.empno
manager's manager's ...	mm	m.mgr = mm.empno

Phrase	Alias to use	Join condition
manager's manager's manager's ...	mmm	mm.mgr = mmm.empno
employee's dept / loc	d	e.deptno = d.deptno
manager's dept / loc	dm	m.deptno = dm.deptno

```

SELECT e.ename, m.ename AS manager, mm.ename AS managers_manager
FROM emp e JOIN emp m ON e.mgr = m.empno JOIN emp mm ON m.mgr = mm.empno;

-- Q161: ename, manager hiredate and job for employees in NEW YORK
SELECT e.ename, m.hiredate, m.job FROM emp e JOIN emp m ON e.mgr=m.empno
JOIN dept d ON e.deptno=d.deptno WHERE d.loc='NEW YORK';
-- Q141: annual salary and loc of employees whose manager is KING
SELECT e.sal*12, d.loc FROM emp e JOIN emp m ON e.mgr=m.empno JOIN dept d ON e.deptno=d.deptno WHERE m.ename='KING';
-- Q159/160 style: 'manager earns less than employee', 'hired before the manager'
... WHERE m.sal < e.sal          ... WHERE e.hiredate < m.hiredate
-- Include KING (who has no manager): use LEFT JOIN emp m ON e.mgr = m.empno

```

Scenario tables: Parent (PK) and Child (FK)

First find the child (the table with the FK). STD has TID (FK) so STD is child; TRN is parent. Doctor has no FK in the given list, so assume PATIENT carries DID as FK. Then the sentence names the table to keep fully: 'for all STUDENTS even if no trainer' = STD LEFT JOIN TRN. 'for all TRAINERS even if no students' = STD RIGHT JOIN TRN. Both = FULL JOIN.

```

-- Q1 students even if no trainer
SELECT s.sname, s.dob, t.tname, t.branch FROM std s LEFT JOIN trn t ON s.tid = t.tid;
-- Q2 all trainers even if no students
SELECT * FROM std s RIGHT JOIN trn t ON s.tid = t.tid;
-- Q3 both
SELECT * FROM std s FULL JOIN trn t ON s.tid = t.tid;
-- Doctor/Patient Q2: all patients even without doctor, address PUNE (filter on kept table)
SELECT * FROM doctor d RIGHT JOIN patient p ON d.did = p.did WHERE p.address = 'PUNE';

```

PART H: DDL, DML, TCL, DCL (Q171-210)

Group	Full form	Commands	Hook	Auto-commit?
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	'CADTR' = structure	Yes
DML	Data Manipulation Language	INSERT, UPDATE, DELETE (MERGE)	'IUD' = data	No (needs COMMIT)
DQL / DRL	Data Query / Retrieval	SELECT	read only	-
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	'CRS' = save or undo	-
DCL	Data Control Language	GRANT, REVOKE	permissions	Yes

```

CREATE TABLE dept2 (deptno NUMBER(2) PRIMARY KEY, dname VARCHAR2(20), loc VARCHAR2(20));
CREATE TABLE emp2 (empno NUMBER(4) PRIMARY KEY, ename VARCHAR2(20), sal NUMBER(8,2),
                    deptno NUMBER(2) REFERENCES dept2(deptno));
CREATE TABLE t5 (a NUMBER, b VARCHAR2(10), c DATE, d NUMBER, e VARCHAR2(5));
RENAME emp TO employees; -- or ALTER TABLE emp RENAME TO employees;
TRUNCATE TABLE salgrade;
DROP TABLE emp2; FLASHBACK TABLE emp2 TO BEFORE DROP; -- get it back from recycle bin
DROP TABLE emp2 PURGE; or PURGE TABLE emp2; or PURGE RECYCLEBIN;
SELECT * FROM RECYCLEBIN; SHOW RECYCLEBIN;

ALTER TABLE employees ADD (gender VARCHAR2(1));
ALTER TABLE dept ADD (c1 NUMBER, c2 VARCHAR2(10));
ALTER TABLE employees DROP COLUMN gender;
ALTER TABLE emp RENAME COLUMN deptno TO dno;
ALTER TABLE emp MODIFY (ename VARCHAR2(50)); -- change datatype / size
ALTER TABLE emp MODIFY (ename NOT NULL); ALTER TABLE emp MODIFY (ename NULL);
ALTER TABLE emp ADD CONSTRAINT pk_emp PRIMARY KEY (empno);
ALTER TABLE emp ADD CONSTRAINT fk_dno FOREIGN KEY (deptno) REFERENCES dept(deptno);

DESC emp;
SELECT constraint_name, constraint_type, search_condition FROM user_constraints
  WHERE table_name = 'EMP';
CREATE INDEX idx_ename ON emp(ename);

```

-- parent first
-- child second
-- no PK, Q171
-- Q172
-- Q173
-- Q174
-- Q175 (gone forever)
-- Q199
-- Q177
-- Q178
-- Q179
-- Q180 (do on dept too)
-- Q209
-- Q210
-- Q206 structure
-- Q198: P=PK R=FK U=Unique C=Check
-- Q205

DML, TCL, DCL

```

INSERT INTO emp (empno, ename, job, sal, deptno) VALUES (9001, 'RAVI', 'CLERK', 1200, 10);
INSERT INTO emp VALUES (9002, 'ANU', 'CLERK', 7902, DATE '2024-01-10', 1300, NULL, 20);
INSERT INTO emp SELECT * FROM emp_backup;
DELETE FROM emp WHERE ename='KING';
UPDATE emp SET sal = 5000 WHERE ename='SMITH';
UPDATE emp SET job='MANAGER' WHERE sal < 1600;
DELETE FROM emp WHERE deptno IN (SELECT deptno FROM dept WHERE loc='NEW YORK');
DELETE FROM emp WHERE sal > (SELECT sal FROM emp WHERE ename='SMITH')
  AND hiredate < (SELECT hiredate FROM emp WHERE ename='SCOTT');
UPDATE emp SET comm = 1000 WHERE mgr = (SELECT empno FROM emp WHERE ename='KING');
UPDATE emp SET job='BOSS', sal=90000
  WHERE deptno IN (SELECT deptno FROM dept WHERE LENGTH(dname) - LENGTH(REPLACE(dname,'A')) = 1);

COMMIT; ROLLBACK; SAVEPOINT s1; ROLLBACK TO s1;
GRANT SELECT, DELETE ON emp TO scott; REVOKE DELETE ON emp FROM scott;
SELECT * FROM user_tab_privs_made; SELECT * FROM user_tab_privs_recd;
-- Q189: as owner (HR) GRANT DELETE ON emp TO scott; then scott runs DELETE FROM hr.emp WHERE sal=3000;
-- Q190: after REVOKE, scott's DELETE fails with ORA-00942 (table or view does not exist) / insufficient privileges

```

-- all columns in order
-- insert from a query
-- Q182
-- Q183
-- Q184
-- Q185
-- Q186
-- Q187
-- Q188
-- TCL
-- DCL
-- check permission list (Q20)

Rules to speak (DDL / DML)

Command	Rule	Hook
CREATE TABLE	Name/column must not start with number or special char (1EMP, @EMP invalid; EMP1 valid). Name must be unique in the schema. Create PARENT before CHILD.	Letter first, parent first
TRUNCATE / DROP parent	Not possible while a child FK refers to it. Remove the connection (drop child or constraint) first.	Child first, parent last
ALTER ADD column	Works on any table if the new column is NULL. NOT NULL column only works on an EMPTY table.	Filled table = NULL only
ALTER DROP column	No restriction.	
ALTER MODIFY datatype	Empty column: any change. Filled column: can only INCREASE size, cannot change type.	Filled = widen only
NULL to NOT NULL	Fails if column already holds NULLs. Not applicable to PK (already NOT NULL).	
Add PK / FK	Only ONE PK per table. FK must point to a PK/unique column of parent (or same table).	One PK, FK to PK

Command	Rule	Hook
INSERT	In child table the FK value must already exist in the parent (or be NULL).	Parent value first
DELETE	Parent row with child rows cannot be deleted until children are removed (unless ON DELETE CASCADE).	
UPDATE	Parent PK can only be changed if no child uses it. Child FK must be set to an existing parent value or NULL.	

PART I: SRF (Single Row Functions) cheat sheet (Q211-240)

Function	Purpose	Example
LENGTH(s)	Number of characters	LENGTH('VIRAT') = 5
UPPER / LOWER / INITCAP	Case change (INITCAP = First Letter Capital)	INITCAP('smith') = Smith
SUBSTR(s, start, len)	Cut part of string. Negative start counts from end.	SUBSTR('SMITH',2,3)=MIT, SUBSTR(s,-2)=last 2
INSTR(s, sub, start, n)	Position of sub string (0 if absent)	INSTR('A B C',' ',1,2) = 4
REPLACE(s, old, new)	Replace every occurrence (no 3rd arg = delete)	REPLACE(ename,'A')
REVERSE(s)	Reverses the string	REVERSE('ABC') = CBA (works in Oracle, MySQL)
CONCAT(a,b) or a b	Join strings (CONCAT takes only 2)	ename ' ' job
LPAD / RPAD(s, n, ch)	Pad to length n	LPAD('7',3,'0') = 007
TRIM / LTRIM / RTRIM	Remove spaces or given char	TRIM(' A ')
MOD(a,b)	Remainder	MOD(10,5) = 0
ROUND / TRUNC / CEIL / FLOOR	Rounding	ROUND(12.57,1)=12.6
NVL(x, y)	If x is NULL use y	sal + NVL(comm,0)
SYSDATE	Today's date and time	SELECT SYSDATE FROM dual
MONTHS_BETWEEN(d1,d2)	Months between dates (d1 later first)	MONTHS_BETWEEN(SYSDATE, hiredate)
ADD_MONTHS / LAST_DAY / NEXT_DAY	Date arithmetic	ADD_MONTHS(SYSDATE,3)
TO_CHAR(d,'fmt')	Date to text. Formats: DD MON MONTH YYYY YY DAY DY	TO_CHAR(hiredate,'DAY')
TO_DATE(s,'fmt')	Text to date	TO_DATE('01-JAN-81','DD-MON-YY')

Mnemonic for string cutting: SUBSTR = 'Slice', INSTR = 'Index/where', REPLACE = 'Swap', LPAD = 'Pad left'. **Half of a name:** LENGTH/2 for first half; the second half starts at LENGTH/2 + 1. **Palindrome:** value = REVERSE(value).

```

SELECT LENGTH('VIRAT') FROM dual; -- Q211
SELECT * FROM emp WHERE LENGTH(ename) = 4; -- Q212
SELECT ename, loc FROM emp e JOIN dept d ON e.deptno=d.deptno WHERE LENGTH(sal)=4; -- Q213 (4 digit sal)
SELECT MOD(10,5) FROM dual; -- Q214
SELECT * FROM emp WHERE MONTHS_BETWEEN(SYSDATE, hiredate) > 513; -- Q216
SELECT REGEXP_REPLACE('MALAYALAM','A','X',1,3) FROM dual; -- Q217 replace 3rd A
SELECT SUBSTR(ename,1,LENGTH(ename)/2) || REVERSE(SUBSTR(ename,LENGTH(ename)/2+1)) FROM emp; -- Q218
SELECT LPAD(SUBSTR('9900199001',7),10,'*') FROM dual; -- Q219 first 6 digits as *
SELECT * FROM emp WHERE LENGTH(ename) - LENGTH(REPLACE(ename,'A')) = 2; -- Q220 exactly 2 A
SELECT d.dname, e.sal FROM emp e JOIN dept d ON e.deptno=d.deptno WHERE ename = REVERSE(ename); -- Q221 palindrome
SELECT * FROM emp WHERE TO_CHAR(sal) = REVERSE(TO_CHAR(sal)); -- Q231 palindrome salary
SELECT ename, sal + NVL(comm,0) AS total FROM emp; -- Q223
SELECT INITCAP(e.ename), INITCAP(d.dname), LOWER(e.job) FROM emp e JOIN dept d ON e.deptno=d.deptno; -- Q226
SELECT ROUND(MONTHS_BETWEEN(SYSDATE, DATE '2000-05-15')) FROM dual; -- Q227 (use your own DOB)
SELECT TRUNC(MONTHS_BETWEEN(SYSDATE, DATE '2000-05-15')/12) AS age FROM dual; -- Q228
SELECT e.ename || SUBSTR(d.dname,1,LENGTH(d.dname)/2) FROM emp e JOIN dept d ON e.deptno=d.deptno; -- Q229
SELECT * FROM emp WHERE ename LIKE '%%' ESCAPE '\'; -- Q230 % as last letter
SELECT e.* FROM emp e JOIN dept d ON e.deptno=d.deptno WHERE SUBSTR(e.ename,1,1) = SUBSTR(d.dname,1,1); -- Q232
-- Q233: add AND SUBSTR(e.ename,-1) = SUBSTR(d.dname,-1)
-- Q234 middle letters vowel: REGEXP_LIKE(SUBSTR(ename,2,LENGTH(ename)-2),'^[AEIOU]+$')
-- Q235 middle word of 'VIRAT ANUSKHA KOHLI'
SELECT SUBSTR(s, INSTR(s,'')+1, INSTR(s,' ',1,2) - INSTR(s,' ') - 1) FROM (SELECT 'VIRAT ANUSKHA KOHLI' s FROM dual);
-- Q236 greeting
SELECT 'HI '||ename||' GOOD MORNING YOUR DESIGNATION IS '||job||' AND SALARY IS '||sal||' YOUR DEPTNO IS '||deptno FROM emp;
SELECT * FROM emp WHERE TRIM(TO_CHAR(hiredate,'DAY')) = 'WEDNESDAY'; -- Q237
SELECT e.*, d.* FROM emp e JOIN dept d ON e.deptno=d.deptno
WHERE TO_CHAR(e.hiredate,'MON') IN ('DEC','JAN','FEB'); -- Q238
SELECT * FROM emp WHERE TO_CHAR(hiredate,'YYYY') IN ('1981','1987','1982'); -- Q239
SELECT TRUNC(MONTHS_BETWEEN(SYSDATE,DOB)/12) AS age, TO_CHAR(DOB,'DAY') AS born_on FROM ...; -- Q240
-- Q225 2nd max salary in each job
SELECT job, MAX(sal) FROM emp e WHERE sal < (SELECT MAX(sal) FROM emp WHERE job = e.job) GROUP BY job;
-- Q224 3rd max details: see Part F DENSE_RANK

```

Note on REVERSE: it is undocumented but works in Oracle SQL. If your lab says it is not available, use a recursive method or tell the interviewer you would use REVERSE (or STRING_AGG/loop in PL/SQL). If a question number or variant differs from yours, keep the same pattern and change only the column or value.

PART J: Differences (Q241-265): one-line answers

Pair	Left	Right
DBMS vs RDBMS	Files, no relations, single user	Tables + keys, constraints, multi-user
CHAR vs VARCHAR	Fixed length, padded	Variable length, saves space
SQL vs NoSQL	Relational, tables, fixed schema, ACID, joins	Non-relational (document, key-value, graph), flexible schema, scales horizontally
Projection vs Selection	Chooses columns (SELECT list)	Chooses rows (WHERE)
WHERE vs HAVING	Filters rows before grouping, no aggregates	Filters groups after GROUP BY, allows aggregates
Alias vs Distinct	Alias = temporary new name for column/table (AS)	Distinct = removes duplicate rows from output
Distinct vs Unique	Query keyword, affects output only	Constraint, prevents duplicates being stored
Primary key vs Foreign key	Unique + not null, one per table, parent	Refers to a PK, duplicates/NULL allowed, many, child
Primary key vs Unique	No NULL, one per table	NULL allowed (Oracle: multiple NULLs), many per table
NULL vs NOT NULL	NULL = unknown/missing (not zero or space)	Constraint that forces a value
Expression vs Alias	Calculation on columns (sal*12)	Name given to the result (AS annual)
Functions vs Operators	Named routines with () (UPPER, SUM)	Symbols/keywords (+, =, AND, IN)
User defined vs Built-in	Written by user (CREATE FUNCTION)	Provided by Oracle (UPPER, SUM)
SRF vs MRF	One result per row	One result for many rows
SQSR vs MRSQ	Sub query returns one row, use = > <	Returns many rows, use IN / ANY / ALL
Sub query vs Joins	Output from one table, inner runs first, can be slower	Output from several tables, usually faster
ALL vs ANY	Condition must hold against every value	Against at least one value
Inner vs Cartesian	Matching rows only (needs ON)	All combinations m x n (no condition)
Natural vs Inner	Auto-joins on same-named columns, common column shown once	You write ON; works with different column names
Outer vs Inner	Keeps unmatched rows (NULL filled)	Drops unmatched rows
DELETE vs TRUNCATE	DML, WHERE allowed, rollback possible, slower, keeps space	DDL, removes all rows, no rollback, faster, resets space
DELETE vs DROP	Removes rows, structure stays	Removes whole table (goes to recycle bin)
TRUNCATE vs DROP	Structure stays, rows removed	Structure and rows removed
FLASHBACK vs ROLLBACK	Restores a DROPPED table from recycle bin	Undoes uncommitted DML (before COMMIT)
ALTER vs UPDATE	DDL, changes table STRUCTURE	DML, changes row DATA
DDL vs DML	Defines structure, auto-commit	Changes data, needs COMMIT
Table vs View	Stores real data physically	Virtual table from a query, stores only the query

PART K: Projects: relationship to table design in 30 seconds

Relationship	How to build	Example
One to Many (1:M)	Put the FK in the MANY side table	Boys 1 - M Girls: girls.bid REFERENCES boys(bid)
One to One (1:1)	FK in child + UNIQUE on that FK	customer - bank: cid UNIQUE REFERENCES customer
Many to Many (M:M)	Third junction table with both FKs; PK = both together	film_actors(fid, aid, PRIMARY KEY(fid, aid))

Order trick: CREATE and INSERT parents first, then children. DROP and DELETE children first, then parents. 'Parent' = the table whose column is referred to (the one with the PK). Optional participation (a boy with no girlfriend, a customer with no account) means: the parent has rows that no child row points to; a child row with NULL FK means 'no parent'.

```
-- Project 1: one-to-one (10 customers, 8 with accounts)
CREATE TABLE customer (cid NUMBER PRIMARY KEY, cname VARCHAR2(30), mail VARCHAR2(40) UNIQUE,
                        mno NUMBER(10), dob DATE, address VARCHAR2(60));
CREATE TABLE bank (bid NUMBER PRIMARY KEY, cid NUMBER UNIQUE REFERENCES customer(cid),
                    bname VARCHAR2(30), acno NUMBER UNIQUE, available_balance NUMBER(12,2));
```

```
-- insert cid 1..10 into customer; bank rows use cid 1..8 only (9,10 have no account)

-- Project 2: one-to-many (6 boys, 18 girls)
CREATE TABLE boys (bid NUMBER PRIMARY KEY, bname VARCHAR2(30), age NUMBER, height NUMBER, type VARCHAR2(20));
CREATE TABLE girls (gid NUMBER PRIMARY KEY, gname VARCHAR2(30), age NUMBER, height NUMBER, type VARCHAR2(20),
    bid NUMBER REFERENCES boys(bid));
-- Plan: boy1 -> 4 girls, boy2 -> 3, boy3 -> 5, boy4 -> 4 (= 16), girls 17 and 18 get bid NULL, boys 5 and 6 get none

-- Project 3 and 4: many-to-many with junction
CREATE TABLE films (fid NUMBER PRIMARY KEY, fname VARCHAR2(100), genre VARCHAR2(50),
    release_year NUMBER(4), budget NUMBER(10));
CREATE TABLE actor (aid NUMBER PRIMARY KEY, aname VARCHAR2(100), dob DATE, nationality VARCHAR2(50));
CREATE TABLE film_actors (fid NUMBER REFERENCES films(fid), aid NUMBER REFERENCES actor(aid),
    role_name VARCHAR2(50), salary NUMBER(10),
    CONSTRAINT pk_fa PRIMARY KEY (fid, aid)); -- composite PK blocks duplicate pairs
-- Project 4 rule: leave 2 books and 3 authors out of BOOK_AUTHOR entirely.
-- Project 4 book: price NUMBER(8,2), bname VARCHAR2(100) NOT NULL
```

Project	Creation order (parent to child)	Relationship hooks
5 Banking	Branches, Customers, Accounts, Employees, Transactions, Loans, Credit_Cards	Customer 1:M Accounts/Loans/Cards; Account 1:M Transactions; Branch 1:M Employees
6 IPL	Team, Coach, Player, Venue, Match, Performance	Team 1:1 Coach (UNIQUE team_id in coach); Team 1:M Player; Match has team1_id and team2_id (both FK to team, which gives M:M teams-matches); Match 1:M Performance
7 College Fest	Department, Organizer, Venue, Student, Event, Participation	Participation is the junction between Student and Event (M:M); CHECK Position IN ('Winner','Runner','Participant')
8 Zomato	Restaurant, Category, Restaurant_Category, Menu, Menu_Item, Customer, Address, Delivery_Person, Orders, Order_Item, Payment, Review	Restaurant_Category = M:M junction; Order_Item PK (order_id, item_id); Orders-Payment 1:1 (UNIQUE order_id); use CHECK and DEFAULT as listed
9 Train	Train, Coach, Passenger, Ticket, Seat_Reservation	Seat_Reservation links Ticket and Coach; CREATE INDEX idx_train ON train(train_name)

```
-- Zomato style constraints you will be asked to write
price          NUMBER(8,2) CHECK (price > 0),
available_status VARCHAR2(3) DEFAULT 'YES' CHECK (available_status IN ('YES','NO')),
registration_date DATE DEFAULT SYSDATE,
phone VARCHAR2(10) UNIQUE,
PRIMARY KEY (order_id, item_id) -- composite PK at the end of CREATE TABLE
```

PART L: 20 Rapid-fire interview answers + last-minute checklist

Question	Short answer
Can WHERE use aggregate functions?	No. Use HAVING, or a sub query: WHERE sal > (SELECT AVG(sal) FROM emp).
Can we use alias in WHERE?	No, WHERE runs before SELECT. It works in ORDER BY.
COUNT(*) vs COUNT(col)?	COUNT(*) counts all rows; COUNT(col) skips NULLs.
Why = NULL fails?	NULL is unknown; comparisons with it are never true. Use IS NULL.
NOT IN with NULL in list?	Returns no rows. Use NOT EXISTS or filter NULLs out.
Find duplicates?	GROUP BY col HAVING COUNT(*) > 1.
Delete duplicate rows?	DELETE FROM emp WHERE ROWID NOT IN (SELECT MIN(ROWID) FROM emp GROUP BY col).
Nth highest salary?	Nested MAX with < (N-1 levels) or DENSE_RANK() = N.
Employees earning more than their manager?	Self join: e.sal > m.sal with e.mgr = m.empno.
Employees with no manager?	WHERE mgr IS NULL (KING).
Dept with no employees?	dept LEFT JOIN emp ... WHERE e.empno IS NULL, or NOT IN / NOT EXISTS.
UNION vs UNION ALL?	UNION removes duplicates, UNION ALL keeps them (faster).
ORDER BY default?	Ascending. DESC for descending. Can use column position (ORDER BY 2).
IN vs EXISTS?	IN compares a list; EXISTS checks if sub query returns any row (good for large inner sets, co-related).
What is a view?	Stored query, a virtual table. CREATE VIEW v AS SELECT ...
What is an index?	Structure that speeds up searches on a column. CREATE INDEX i ON emp(ename).
What is normalization?	Splitting tables to remove redundancy: 1NF atomic values, 2NF no partial dependency, 3NF no transitive dependency.

Question	Short answer
ACID?	Atomicity, Consistency, Isolation, Durability: properties of a transaction.
DELETE without WHERE?	Deletes all rows, but can be rolled back. TRUNCATE cannot.
Order of execution?	FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

Last-minute checklist (before you speak or write any answer)

Step	Ask yourself
1	Which table gives the OUTPUT columns? Which table holds the CONDITION?
2	One table: plain WHERE. Condition from a person's row: sub query Case 1. Condition from another table: Case 2 or join.
3	Output needs columns from 2 tables: join. Needs comparison with the same table's other row (manager, 'before X'): self join or sub query.
4	Words 'each / wise': GROUP BY. 'if / at least' on groups: HAVING. 'Nth': nested MAX/MIN or DENSE_RANK.
5	Words 'even if / not assigned': outer join. Check which table must be kept, and whether an IS NULL filter is needed.
6	Check NULLs, date boundaries (after 81 means after 31-DEC-81), quotes on text values (UPPERCASE: 'KING'), semicolon at the end.